

SAKI: Shared Adaptive KV-Instance Control under a Fixed Compaction Budget

Qichu Tian

Xi'an Jiaotong University
mizukiqwq@stu.xjtu.edu.cn

Shipeng Zhao

Xi'an Jiaotong University
zhaoshipeng@stu.xjtu.edu.cn

Heng Chen

Xi'an Jiaotong University
hengchen@xjtu.edu.cn

Bo Wang

Xi'an Jiaotong University
Aroom@stu.xjtu.edu.cn

ABSTRACT

Multi-tenant deployments often co-locate many RocksDB instances on one host. Each tenant has an independent LSM pipeline, but flushes and compactions draw from a shared background I/O and compaction budget. When write pressure drifts, equal per-instance allocation can leave service stranded at quiet tenants while tenants creating compaction debt suffer high write-tail latency. We frame this as fixed-budget placement: moving the existing background budget toward tenants where compaction pressure is forming.

SAKI is an external controller for this placement problem. In its conserved rank-and-assign framework, the controller observes per-tenant demand and pressure symptoms, ranks tenants without oracle labels or workload forecasts, and reallocates per-tenant background budgets while preserving the aggregate budget. A demand-observability model explains when demand-aware ranking keys track bounded drift within one window and why pressure-only ranking can suffer stale-debt lag. SAKI requires no changes to RocksDB's compaction algorithm or tenant application code. Across five 16-tenant epoch-level drift trials, SAKI reduces high-pressure write P99 by 11.7%, raises high-pressure throughput by 38.1%, and lowers compaction write bytes by 21.0% relative to equal static allocation, while keeping total throughput roughly neutral. In a continuous embedded RocksDB harness with long-running tenants and runtime actuation through `RateLimiter::SetBytesPerSecond()`, SAKI reduces high-pressure write P99 by 27.0%, reduces P999 by 47.1%, raises high-pressure throughput by 26.1%, and lowers normalized bytes/write by 11.9%. Stronger baselines, ranking replay, ranking-key perturbations, and mechanism checks attribute these gains to online placement rather than extra capacity, static skew, ranking-key fragility, or unrelated throttling. Public-trace audits identify what future production traces must preserve for controller validation. The results show that shared compaction budgeting is a practical host-level control point for multi-tenant RocksDB deployments under fixed background budgets.

PVLDB Reference Format:

Qichu Tian, Heng Chen, Shipeng Zhao, and Bo Wang. SAKI: Shared Adaptive KV-Instance Control under a Fixed Compaction Budget. PVLDB, 20(1): XXX–XXX, 2027.

doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/M1zukiqwq/PVLDB2027-SAKI-OpenSource>.

1 INTRODUCTION

Modern data-serving systems often place many storage-engine tenants on one host. Each tenant may own a separate RocksDB instance, data directory, configuration, and foreground workload, yet their flushes and compactions draw from the same finite background CPU and storage bandwidth. Per-instance controls are useful, but they do not decide how a fixed host budget should be divided across competing instances.

This placement decision becomes a performance problem when demand drifts. Equal static allocation leaves capacity assigned to quiet tenants; frozen skew becomes stale when the high-pressure set moves; and RocksDB's per-instance auto-tuned rate limiter adapts within a configured ceiling but cannot move budget across instances. The resulting compaction-debt mismatch appears as high write-tail latency and lower completion rates for tenants currently creating debt.

We study the host-level placement decision directly. Under a fixed aggregate background compaction/I/O budget, can an online controller use runtime observations to reallocate per-tenant budgets across co-located RocksDB tenants, improving the tenants currently under write pressure without oracle labels, workload forecasts, or extra host capacity? The question is not whether more compaction capacity helps, but whether the same capacity can be placed better than fixed fair sharing.

Several close directions leave this placement point open. General multi-tenant storage controllers enforce lower-layer I/O shares or SLOs, while recent LSM-oriented systems either change a single engine, move flush or compaction into a disaggregated service or DPU path, or build isolation into a different multi-tenant engine boundary [4, 6, 8, 25, 28, 32]. SAKI instead targets a common operational shape: many separate, unmodified RocksDB instances on one host, with only a conserved background budget to place across them as demand drifts.

SAKI is an external controller for this placement problem. It observes each tenant over a control window, ranks tenants using offered write demand and compaction-pressure symptoms, and

licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

applies the next-window budget assignment while conserving the aggregate budget. This conserved rank-and-assign framework separates the placement invariant from the ranking key. Its key condition is demand observability: when offered demand separates currently high-pressure tenants from the rest, demand-aware ranking keys track bounded drift within one control window, while pressure-only ranking can chase stale debt. SAKI sits outside RocksDB; in the continuous harness, it changes runtime rate-limiter budgets through `RateLimiter::SetBytesPerSecond()`.

The evaluation follows an attribution chain. It first asks whether SAKI improves the target tenants under the same aggregate budget, using both epoch-level and continuous RocksDB experiments. It then tests alternative explanations, including static skew, tenant-local autotuning, stale-debt scoring, lower-layer I/O weights, and foreground throttling. Finally, ranking-margin replay, ranking-key perturbations, workload-matrix experiments, collateral measurements, liveness checks, and public-trace audits define where the evidence applies.

In a 16-tenant epoch-level workload, five trials show that SAKI reduces high-pressure write P99 by 11.7%, raises high-pressure throughput by 38.1%, and lowers compaction write bytes by 21.0% relative to equal static allocation, with roughly neutral total throughput. In a continuous embedded RocksDB harness with runtime actuation, five trials show that SAKI reduces high-pressure write P99 by 27.0%, reduces P999 by 47.1%, raises high-pressure throughput by 26.1%, and lowers normalized bytes/write by 11.9%. The tradeoff is visible: lower-pressure tenants may absorb part of the cost, so SAKI is a targeted reallocation policy.

The paper makes five contributions:

- (1) It frames multi-tenant RocksDB compaction as a fixed-budget host-level reallocation problem, distinct from single-instance compaction-policy design.
- (2) It presents SAKI, a conserved rank-and-assign controller that sets per-tenant runtime budgets from previous-window observations without oracle labels, workload forecasts, or tenant application changes.
- (3) It identifies the demand-observability condition that lets a demand-aware ranking key track bounded drift, and contrasts it with the stale-debt lag of pressure-only ranking.
- (4) It provides epoch-level and continuous evidence that online reallocation improves high-pressure write latency and throughput under bounded drift while keeping total throughput close to static allocation.
- (5) It separates SAKI's effect from fixed skew, per-instance local autotuning, ranking-key choice, workload variation, and lower-layer I/O control, while making collateral costs and validation boundaries explicit.

Together, these results show that shared host-level compaction budgeting is a practical control point for multi-tenant RocksDB write performance under bounded drift.

2 BACKGROUND AND MOTIVATION

SAKI targets co-located RocksDB instances whose background work shares finite host resources. RocksDB exposes useful local controls, but not the host-wide placement decision: how that fixed

service budget should be placed across instances as tenants' write pressure changes at different times.

2.1 LSM Background Work

The log-structured merge-tree makes foreground writes inexpensive by buffering updates and later merging them into on-disk components [20]. Compaction rewrites sorted files, removes obsolete entries, controls level sizes, and bounds read cost. This design gives high write throughput but turns recent foreground writes into future CPU and I/O demand. That deferred demand becomes a host-level problem when several engines share one finite background budget; surveys and recent microbenchmarks emphasize compaction, amplification, and buffering as central LSM trade-offs [10, 14].

RocksDB exposes per-instance background jobs, rate limiters, statistics, properties, and event listeners [16]. SAKI uses them as the measurement and actuation surface for the host-level placement problem.

2.2 From Tenant-Local State to Host-Level Debt

We use *compaction debt* as an operational term for deferred background work and its foreground consequences. Pending compaction bytes, L0 files, flush/compaction events, write-tail latency, and completion gaps are observable but incomplete symptoms that help identify tenants currently creating deferred maintenance.

The cross-tenant aspect is the key. Under a fixed aggregate budget, equal allocation leaves capacity assigned to quiet tenants whenever only a subset is write-heavy. Frozen skew helps only while the workload resembles the phase it was tuned for. Section 7 tests this mismatch between where the budget sits and where debt is being created.

Compaction service has delayed effects: too little service may surface later as L0 pressure, flush backlog, write slowdown, or tail latency; too much service for a quiet tenant consumes capacity that could reduce risk elsewhere. The evaluation therefore reports high-pressure latency and throughput together with total throughput and collateral impact on lower-pressure tenants.

2.3 Why Existing Directions Do Not Settle the Question

Prior LSM work changes single-engine policies or storage layout: Monkey, Dostoevsky, WiscKey, PebblesDB, and SILK improve what an engine does given an assigned service budget [1–3, 13, 21]. SAKI asks where a fixed host budget should be placed across independent engines.

Closest multi-tenant directions still leave the host-level placement question open. Lower-layer storage controllers enforce shares elsewhere, while recent LSM-oriented systems move admission, fairness, compaction, or flush work into a different engine boundary or service location [4, 6, 8, 25, 28, 32]. SAKI studies the remaining case: a controller that observes tenant-local LSM demand and maintenance symptoms and reallocates a conserved budget across separate unmodified RocksDB instances through per-instance runtime knobs.

3 PROBLEM DEFINITION

We study a host with tenants $T = \{1, \dots, N\}$. Each tenant is an independently configured RocksDB instance with its own memtables, WAL, SST files, flush path, compaction path, and foreground workload. Tenants do not share logical key spaces, but their background work draws from shared host resources. Time is divided into control windows $w = 1, 2, \dots$. In each window, the controller assigns tenant i a background budget $b_i(w)$. The unit of this budget is harness-dependent: epoch runs change rate limits and background-job settings between epochs, while continuous runs keep tenants alive and change the rate-limiter budget through `RateLimiter::SetBytesPerSecond()`.

The aggregate budget is fixed. If B is the host budget, a valid assignment must satisfy $\sum_i b_i(w) = B$, subject to the discrete budget tiers exposed by the implementation. Without this conservation rule, a controller could improve high-pressure tenants simply by spending more host capacity.

The controller is online. At the start of window w , it may use only observations from windows up to $w - 1$. The observation vector $x_i(w - 1)$ can include demand signals such as offered write demand and completed write throughput; pressure symptoms such as completion gap, write-tail latency, pending-compaction proxies, and L0 pressure; background-work signals such as compaction bytes and flush/compaction events; read-side measurements; and the prior assignment. It may not include the next high-pressure set, future phase id, or evaluator-only tier labels. SAKI also does not observe a single ground-truth scalar called compaction debt; it observes symptoms of deferred work and foreground pressure.

The objective is targeted relief for tenants currently under high write pressure, measured by write-tail latency and write throughput, while preserving attribution to online reallocation under a fixed budget. Because reallocation is a tradeoff, collateral effects must also be visible: the evaluation reports mid- and low-pressure effects, total throughput, and liveness failures. Each decision therefore chooses $b_i(w)$ from implementable budget levels using $x_i(w - 1)$, conserving B under delayed feedback.

Several constraints sharpen the scope. Policies cannot read workload labels or future drift schedules. Continuous runs actuate only the runtime rate limiter, rather than every RocksDB option. Assignments occur at window boundaries, so very fast drift may outrun a previous-window controller. SAKI is therefore not a new compaction algorithm, an in-engine global scheduler, or an oracle classifier. The definition leaves the ranking rule open; the next section instantiates this online fixed-budget problem with a conserved rank-and-assign framework and a transparent demand-aware ranking key.

4 SAKI DESIGN

SAKI is a host-level closed-loop controller that implements a conserved rank-and-assign framework (Figure 1). The framework is the method contribution: it preserves a fixed-budget invariant and assignment template while allowing different observable ranking keys to order tenants. Each window, SAKI observes each tenant’s demand and pressure signals from window w , ranks tenants from those observations, assigns high/mid/low budget tiers for window $w + 1$, and verifies conservation before actuating. The one-window

delay is intentional: signals from window w must remain informative for window $w + 1$.

4.1 Conserved Rank-and-Assign Framework

The framework decouples a budget-conserving assignment template from the ranking key used to order tenants. The control loop is intentionally compact:

Input: $\{x_i(w)\}_{i=1}^N, B, b_H, b_M, b_L, H, L$
Key: $s_i(w) \leftarrow k(x_i(w))$
Rank: sort tenants by descending $s_i(w)$
Assign: top $H \rightarrow b_H$, bottom $L \rightarrow b_L$, others $\rightarrow b_M$
Check: $\sum_i b_i(w + 1) = B$
Actuate: apply $b_i(w + 1)$ for the next window.

The tiered assignment makes the policy auditable. We distinguish *high-pressure* tenants, which the workload currently stresses, from *high-budget* tenants, which SAKI assigns to b_H . A tier configuration is budget-conserving when $H(b_H - b_M) = L(b_M - b_L)$; the continuous main setting uses $4(11 - 6) = 4(6 - 1)$, yielding the same 96 MB/s aggregate budget as sixteen all-mid tenants. Other tier counts or continuous allocations can use the same rule; we fix auditable tierings to isolate the placement mechanism within each experiment family.

The ranking key only chooses the ordering used by the framework; it is not the mechanism contribution by itself. This separation keeps the paper’s claim on fixed-budget online placement: the assignment template conserves the host budget, and the ranking key provides an auditable way to decide where that conserved budget goes next. The reference key below is intentionally simple so the evaluation attributes gains to placement under conservation, not to an opaque learner, hidden tuning process, or extra capacity.

4.2 Observations and Labels

The observation vector is tenant-local, but the decision is host-wide. In the continuous harness, $x_i(w)$ includes offered write/read rates, completed operations, write latency percentiles, compaction and flush bytes from an event listener, estimated pending compaction bytes, L0 files, memtable bytes, delayed-write rate, write-stop status, and the current assignment. These are runtime measurements rather than workload labels.

The experiment generator retains each tenant’s synthetic workload tier so the evaluation can compute high-pressure benefit and high-set overlap. The controller does not read that tier, a future phase identifier, or the next high-pressure set. Offered write demand is an observed admission-side signal from the previous window, rather than a prediction.

4.3 Reference Ranking Key and Family

The controller needs a ranking key to instantiate the conserved assignment template. We use a demand-aware reference key, not a learned optimizer. Its role is to expose the placement mechanism with measurements that an operator can inspect and replay. Let D_i be offered write QPS, C_i completed write QPS, $G_i = \max(0, D_i - C_i)$ the completion gap, B_i the compaction output in MB, P_i pending compaction bytes in MB, and L_i the number of L0 files capped at 6.

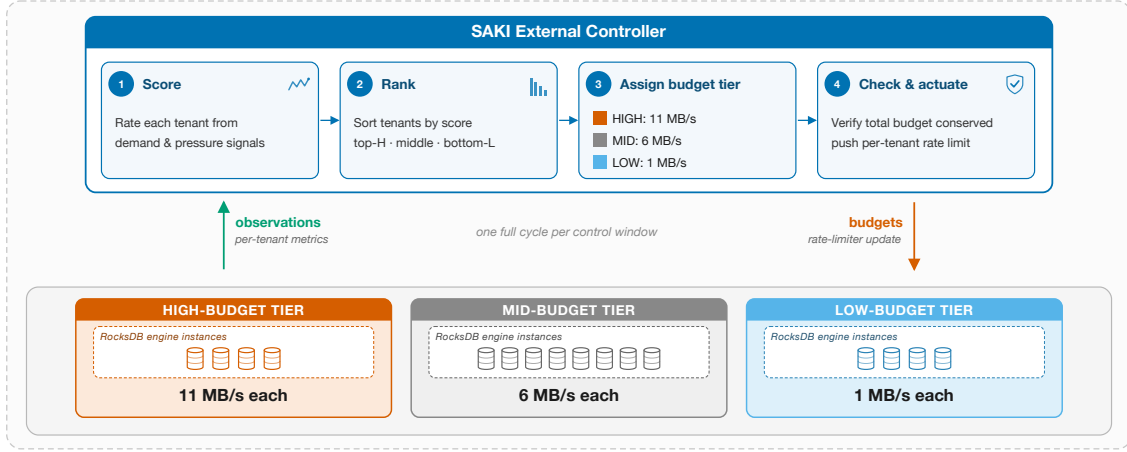


Figure 1: SAKI conserved rank-and-assign loop. The external controller ranks tenants from previous-window demand and pressure signals, assigns high/mid/low budget tiers, checks aggregate-budget conservation, and actuates per-tenant runtime rate limits.

The bounded tail term is

$$T_i = \min(P99_i/1000, 80) + \min(P999_i/5000, 40),$$

where latency is measured in microseconds. In demand mode, SAKI assigns tenant i the key

$$s_i = 6.00D_i/1000 + 0.60G_i/1000 + 0.05T_i + 0.02B_i + 0.02P_i + 0.30L_i.$$

The constants in this reference key are not a universal optimum. They encode one auditable ordering in which offered write demand is the anchor and bounded maintenance symptoms—completion gap, tail latency, compaction output, pending bytes, and L0 pressure—break ties or refine near-boundary tenants. The intended invariant is ordering margin, not exact numeric weights: residual terms should not overturn a clearly separated demand signal. The evaluation treats ranking-key perturbations as placement-family robustness checks; in the main continuous workload, the demand term alone reproduces the reference key’s top-four assignment in all 35 post-warmup decisions.

For portability, a deployment should fix the conserved tier template first, normalize observable signals, and bound residual terms so they refine near-boundary tenants rather than overturn a declared offered-demand separation.

The ranking key is fixed and auditable, but SAKI remains online: observations, ranking, and assignment are recomputed every window. Keeping the key fixed separates the placement question from learning a ranking policy under delayed, partially counterfactual feedback. The relevant design space is therefore a family of demand-aware ranking keys whose primary term is offered write demand and whose secondary pressure terms are bounded refinements. Section 7 tests demand-only replay, perturbed demand-aware keys, and pressure-only negative controls under the same template to distinguish tracking from stale-debt lag.

The epoch harness uses a coarser, pressure-oriented key because each epoch exposes clearer symptoms. The continuous harness is stricter: decisions are made every short window while tenants stay

active. Demand mode is the main continuous policy; pressure and hybrid modes serve as ablations.

4.4 Actuation and Stability

The actuation path matches the harness. Epoch-level runs can change background-job and rate-limit settings between epochs. Continuous runs keep each tenant active and adjust only the runtime rate-limiter budget; `max_background_jobs` remains fixed at process launch. Section 5 gives the specific runtime API.

SAKI’s framework-level stability rests on budget conservation, bounded residual terms, tiered assignment, a low-tier floor, and one-window updates. We evaluate this envelope through high-set overlap, adaptation lag, total throughput, bytes/write, and tier-level collateral effects. Production deployments could add hysteresis, cooldowns, minimum-dwell rules, or SLO-weighted floors. Per window, the controller aggregates N tenant observations, sorts tenants by key, validates the conserved budget, and issues N budget updates. The control-plane work is $O(N \log N)$ per window and stays outside the foreground write path; Section 7 includes larger-inventory liveness runs that isolate this scaling question from the headline performance claim.

4.5 Analytical Motivation

The design choices in the preceding subsections are motivated by a simple analytical model. The model is not an optimizer for ranking constants; instead, it isolates the tracking condition that a ranking key must satisfy and explains the experimental divergence between demand-mode and pressure-mode controllers reported in Section 7.

Model. Let tenant i have offered write demand $d_i(w)$, completed write rate $c_i(b)$ under budget b , and normalized compaction debt $q_i(w)$ in equivalent writes. Budget tiers satisfy $b_H > b_M > b_L$ with conservation $Hb_H + (N - H - L)b_M + Lb_L = B$. Backlog evolves as

$$q_i(w + 1) = \max(0, q_i(w) + T(d_i(w) - c_i(b_i(w)))),$$

where T is the window length. We call tenant i *high-pressure* when $d_i(w)$ exceeds the service rate achievable under b_M ; the deficit under b_L is larger still.

Why static allocation can fail. Under equal allocation $b_i = B/N$, if a high-pressure tenant satisfies $d_i > c_i(B/N)$, then q_i grows without bound until a write stall or SLO violation occurs. This is the baseline problem: equal sharing allocates the fixed budget without regard to which tenants are currently creating debt.

Proposition 1 (demand-aware one-window tracking). Consider a ranking key in the rank-and-assign framework whose ordering can be written as $s_i = \alpha D_i(w) + \rho_i(w)$, where $D_i(w)$ is the observed offered demand and the residual term $\rho_i(w)$ is computed from bounded pressure and maintenance signals. Suppose the high-pressure set to be served has at most H tenants and remains active into the next control window. If, in window w , every such tenant has offered demand at least Δ above every non-high tenant and

$$\alpha\Delta > \max_{i,j} |\rho_i(w) - \rho_j(w)|,$$

then the framework places those tenants in the high-budget tier in window $w + 1$.

The proof is immediate from the ordering condition: for any high-pressure tenant i and non-high tenant j , $s_i - s_j > 0$, so the top- H rank contains the high-pressure set. The one-window delay is unavoidable because the controller only observes $D_i(w)$ after window w completes. The condition is sufficient, not universal: in the continuous main trials, offline replay shows that the demand term alone produces the same top-four assignment as the reference ranking key in all 35 post-warmup decisions.

Proposition 2 (stale-debt lag for pressure-only ranking). Consider a monotone pressure-only key that ranks tenants by backlog or maintenance signals but does not observe offered demand. Immediately after a workload drift, the key cannot distinguish an exiting tenant with historical debt from an entering tenant whose current demand is forming fresh debt. If the exiting tenant starts the post-drift period with residual pressure equivalent to q writes above the entrant, and the extra drain rate gained by removing its high-budget slot is at most $\Delta c = c(b_M) - c(b_L)$, then the stale high-budget assignment can persist for $\Omega(q/(T\Delta c))$ windows under a monotone backlog-priority rule.

This is not a convergence theorem for all controllers; it is a structural ambiguity in the pressure-only ranking family. It predicts the measured divergence between ranking keys: demand mode tracks with overlap 3.00/4 and zero max lag, while pressure-only mode has overlap 1.86/4 and max lag 5 windows. The constants remain a deployment choice, but the demand-observability condition is the mechanism that separates tracking from stale-debt chasing.

5 IMPLEMENTATION

SAKI is an external Python controller plus two experiment harnesses around unmodified RocksDB instances. The epoch harness tests fixed-budget placement between workload phases; the continuous harness tests the same loop through a live RocksDB rate limiter while tenants keep running. In both paths, the controller chooses high/mid/low assignments, the harness applies them through ordinary RocksDB-facing APIs, and measurements are recorded at the controller’s granularity.

5.1 Epoch Harness

The epoch-level path uses `db_bench` as the tenant process. A trial creates one database directory per tenant, runs a prefill phase, and then executes a sequence of workload epochs. Each epoch assigns tenants to high, mid, and low demand tiers, launches one `db_bench` process per tenant, parses the logs, and uses the resulting observations for the next epoch’s allocation decision. Because every epoch is a fresh invocation, this harness can vary both `max_background_jobs` and `rate_limiter_bytes_per_sec` at epoch boundaries. The driver checks that the resulting job and rate-limit settings match the static fair-share budget before launch.

The parser extracts per-tenant runtime, exit status, operation rate, latency histograms where available, maintenance bytes, keys and bytes written, and compaction CPU time. These rows give clean mechanism evidence because all policies run comparable foreground work under the same aggregate budget; runtime control is tested by the continuous harness.

5.2 Continuous Harness

The continuous path is a custom C++ embedded RocksDB harness. Each tenant is a long-running process linked against RocksDB. The Python driver advances through fixed windows, computes workload tiers and policy assignments, validates the aggregate byte budget, and atomically refreshes one control file per tenant. Tenants keep using the last installed budget until a newer value appears, so a missed update cannot create extra host budget; production controllers would also fall back to all-mid allocation when inventory or metrics are incomplete.

Runtime actuation uses RocksDB’s rate limiter. At startup, each tenant creates a `NewGenericRateLimiter` with the initial budget and installs it before opening the database. During the run, a control thread rereads the budget and calls `RateLimiter::SetBytesPerSecond()` on the same rate-limiter object. This is the main continuous actuation path. Background-job count is a launch-time option and remains fixed for the tenant process. Diagnostic userspace throttling exists in the harness but is not the mechanism used for the reported continuous result.

The rate limiter is a RocksDB-facing background-I/O control rather than an application-level admission throttle. It shapes flush and compaction paths; SAKI does not directly sleep foreground reads or writes. Foreground latency changes indirectly through LSM maintenance service, making the continuous result a maintenance-placement result. In the five main trials, limiter, compaction, and flush byte deltas are larger in high-budget windows than in low-budget windows. A homogeneous four-tenant microstudy sweeping $B \in \{1, 3, 6, 11\}$ MB/s confirms the path: measured limiter traffic is monotone (1.00, 2.96, 5.70, 7.28 MB/s), compact-plus-flush output tracks it (coverage 0.99–1.02), and a no-limiter sanity run reports zero limiter bytes while maintenance continues.

The embedded harness records per-window write and read operation counts, throughput, and latency percentiles. A custom `EventListener` accumulates compaction and flush counters. At window boundaries the harness emits deltas and RocksDB properties such as pending compaction bytes, L0 files, memtable bytes, delayed-write rate, and write-stop status. These measurements feed both the online ranking key and the post-run analysis.

5.3 Analysis

The analysis scripts consume JSON payloads produced by the drivers. For continuous runs, the analyzer merges tenant metrics, allocation history, process exit codes, commands, run parameters, and summary counters. It computes write P99/P999 and throughput for high-pressure tenants, total throughput, compaction output bytes, bytes/write, high-set overlap, adaptation lag, and failed-tenant counts. The same pipeline emits the paper-table summaries and the time-series data used for the evaluation figures. This keeps the manuscript numbers tied to generated artifacts rather than manual transcription.

6 EXPERIMENTAL METHODOLOGY

The experiments test fixed-budget placement, then narrow the mechanism through runtime actuation, stronger baselines, ranking-key checks, cgroups, rate-limiter coverage, collateral accounting, liveness, and trace qualification. Budget conservation is enforced within each experiment family: epoch-level and public-trace qualification runs use an 11/7/3 MB/s, 112 MB/s contract, while continuous runtime runs use an 11/6/1 MB/s, 96 MB/s contract. The contracts differ because the harnesses expose different actuation surfaces, but every comparison holds the relevant family budget fixed.

6.1 Workloads

The epoch-level anchor is EPOCH-DRIFT16. It uses 16 co-located tenants, three demand tiers, eight epochs, and gradual movement of a four-tenant high-pressure set. Tenants are prefilled before the measured epochs.

The continuous anchor is RUNTIME-DRIFT16. It also uses 16 tenants and three tiers, but runs tenants as long-lived embedded RocksDB processes for eight 20-second windows. The high-pressure set moves by one tenant per window, so the experiment tests online adaptation at a rate the controller can plausibly follow. The continuous anchor uses 11/6/1 MB/s high/mid/low budgets with a 4/8/4 split, preserving a 96 MB/s aggregate against an all-mid static baseline. Section 7 reports window-length sensitivity, ranking-key perturbations, offline replay, and a longer-horizon sanity run around this anchor.

We also run a local sensitivity sweep around EPOCH-DRIFT16, changing drift speed and high-write pressure. Each new point starts with two static/SAKI screening trials; stress-side variants are promoted to five trials under pre-committed criteria.

Around RUNTIME-DRIFT16 we run a controlled workload matrix that perturbs one axis at a time: VALUE-4K uses 4 KB values, READ-3x triples per-tier point-read QPS, FAST-DRIFT moves the high-pressure set at 1.5 tenants per window, and TENANT-8 uses a smaller 8-tenant cluster with two high-pressure tenants. Per-tenant budgets are preserved so the same-budget contract holds; for TENANT-8 the aggregate scales linearly. The matrix uses the unchanged paper policy and promotes variants under the same screening rule. Supplemental larger-inventory and trace-shaped stresses are reported only as liveness/tracking and coverage checks; FAST-DRIFT and TENANT-8 are local sensitivity, not a parameter search.

6.2 Policies

static is the main fair baseline: every tenant receives the same share of the aggregate background budget for that experiment family. SAKI is the online policy: it assigns high, mid, and low tiers from previous-interval observations while preserving that aggregate budget. The main continuous SAKI policy uses the demand-aware reference ranking key.

static_biased is a frozen-skew baseline for the epoch workload. It uses the same high/mid/low budget tiers, but freezes the initial-stage assignment for all later epochs. It models an operator who pre-tunes a reasonable non-equal allocation and then does not adapt as the high-pressure set drifts.

static_autotuned is the strict same-budget built-in RocksDB baseline. It uses exactly the same per-tenant ceilings as static. In the epoch configuration this is two background jobs and 7 MB/s per tenant, for a 32-job and 112 MB/s aggregate budget across 16 tenants. Each tenant enables `-rate_limiter_auto_tuned=1`, so adaptation stays local to that RocksDB instance and remains within the configured per-instance range. This baseline asks whether per-instance local rate adaptation can replace the outer host-level placement decision; we keep the ceilings identical to static. For the sensitivity sweep, static_autotuned is added only to the promoted higher-pressure point.

oracle_tiered is a diagnostic reference using true tier information. unlimited is a capacity reference, not a same-budget competitor.

6.3 Platform

All experiments run on a single Linux machine with 80 logical cores (Ubuntu, kernel 6.17). Storage is a local SSD. Each RocksDB tenant writes to its own data directory on the same device. The epoch-level harness uses `db_bench 8.9.1` from unmodified Ubuntu .deb archives; the continuous harness links against the same RocksDB version. No other major I/O-intensive processes run during measured trials.

6.4 Metrics and Aggregation

Primary metrics are write P99/P999 latency and write throughput for high-pressure tenants, the foreground symptoms SAKI is meant to improve by moving background service. Total throughput shows whether the host collapses aggregate work under the fixed budget. Tier-wise latency and throughput expose the redistribution cost that a targeted policy can impose on lower-pressure tenants. The continuous rate limiter is background-I/O control: it shapes RocksDB flush/compaction I/O, so foreground changes reflect maintenance placement rather than direct sleeps on application reads or writes.

Epoch-level efficiency reports compaction write bytes. In continuous runs SAKI changes foreground completion, so the main compaction efficiency metric is normalized bytes/write; absolute compaction bytes are reported but not the main efficiency claim. Epoch runs execute comparable foreground work per trial, so absolute background bytes are directly interpretable; continuous runs may complete more foreground writes under SAKI and need per-write normalization.

Adaptation is measured by high-set overlap (high-pressure tenants assigned the high-budget tier after warmup) and adaptation lag in control windows. Ranking robustness is measured by replaying the previous-window observations that fed each online decision and comparing the resulting top- H set across ranking keys using exact-match count and Jaccard similarity. Liveness is measured by failed tenant processes. The main epoch and continuous results aggregate five independent trials. Percent changes are computed relative to static within each trial before aggregation. The paper reports mean, range, standard deviation, and confidence intervals. Sensitivity rows with two trials report a mean and range; five-trial rows report 95% CI half-widths. Raw and summarized artifacts are recorded under `remote-results/`, and figures use real result files from the figure data manifest.

6.5 Scope

The methodology supports a constrained claim: fixed-budget online reallocation in two controlled RocksDB drift regimes. The built-in local rate-limiter baseline is covered under the same aggregate budget because it tests whether tenant-local adaptation can replace the host-level rank-and-assign loop. OS/cgroup I/O controls are included for a different reason: they can enforce weights or throttles below RocksDB, but they do not observe LSM debt signals or decide which engine should receive compaction service. We include a 5-trial aligned cgroup-v2 baseline at the same scale as the epoch main result as supplemental mechanism evidence, not as a repeated main baseline. The controller inside the cgroup harness uses a simplified pressure ranking key; it is not the main SAKI controller used in the primary RocksDB experiments.

7 EVALUATION

The evaluation asks whether fixed-budget placement improves high-pressure tenants, whether the effect survives stronger same-budget alternatives, and where workload coverage and collateral costs bound the claim. Unless stated otherwise, percentages are online SAKI versus equal static allocation; lower latency or bytes/write and higher throughput are improvements. “High” denotes the workload’s high-pressure tenants, and high-set overlap measures how often they receive SAKI’s high-budget assignment. All reported RocksDB and cgroup runs completed with zero failed tenants.

7.1 Fixed-Budget Placement Improves High-Pressure Tenants

The epoch-level harness isolates placement by changing RocksDB settings only between epochs. Across five repeated 16-tenant drift trials, SAKI reduces high-pressure write P99 by 11.7%, raises high-pressure throughput by 38.1%, and reduces compaction write bytes by 21.0% (Table 1). Total throughput remains roughly neutral: the mean is -1.7%, and its CI crosses zero. All five trials improve high-pressure write P99, high-pressure throughput, and compaction bytes; mean high-set overlap is 3.14/4, so the policy places high budget near the moving high-pressure set without using oracle labels.

The continuous harness tests the stricter deployment boundary: tenants remain alive, workload phases drift every 20 seconds, and SAKI changes only the runtime rate-limiter budget through

Table 1: Epoch-level main result, EPOCH-DRIFT16 (online vs. static).

Metric	Mean	Range	95% CI
High write P99	-11.7%	[-17.2,-6.2]	[-18.0,-5.5]
High throughput	+38.1%	[+34.2,+42.1]	[+34.4,+41.7]
Total throughput	-1.7%	[-4.7,+2.1]	[-4.9,+1.4]
Compact bytes	-21.0%	[-24.3,-18.4]	[-23.6,-18.4]
Mean high overlap	3.14/4	[3.14,3.14]	–

Table 2: Continuous main result, RUNTIME-DRIFT16 (five trials, online vs. static). CI half-widths in pp.

Metric	Mean	Min	Max	95% CI
High write P99	-27.0%	-39.2%	-12.0%	±12.5pp
High write P999	-47.1%	-54.0%	-39.3%	±8.8pp
High write throughput	+26.1%	+21.5%	+28.2%	±3.5pp
Total throughput	+4.3%	+1.3%	+6.8%	±3.2pp
Compact bytes	-1.9%	-5.8%	-0.1%	±2.9pp
Bytes/write	-11.9%	-13.5%	-9.1%	±2.3pp
Mean high overlap	3.00/4	3.00	3.00	–

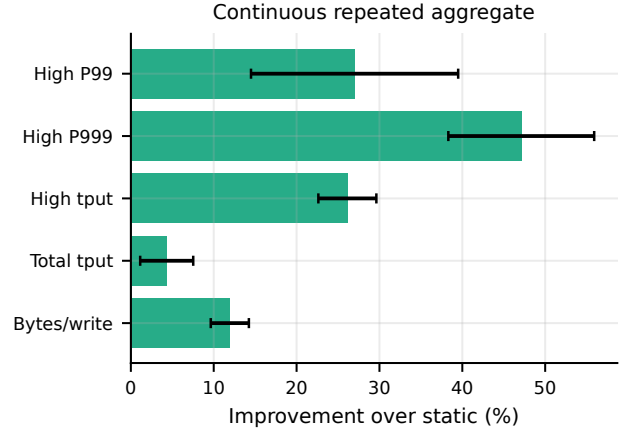


Figure 2: Continuous repeated aggregate. Each bar is normalized so positive means improvement over equal static; error bars show 95% CI half-widths.

`RateLimiter::SetBytesPerSecond()`. Table 2 and Figure 2 summarize five RUNTIME-DRIFT16 trials. SAKI reduces high-pressure write P99 by 27.0%, P999 by 47.1%, and raises high-pressure write throughput by 26.1%. Because SAKI completes more foreground writes, normalized bytes/write is the main continuous efficiency metric; it falls by 11.9%, while absolute compact bytes change only -1.9%. The continuous result shows the placement effect is not an epoch artifact: the same conserved-budget loop works while tenants stay alive and only the RocksDB rate-limiter budget changes.

7.2 Mechanism Checks and Alternative Controls

A fixed-budget improvement could be an artifact of a weak baseline, a stale or fragile ranking key, a non-maintenance throttle, or a lower-layer control that already does the work. The checks in this subsection separate those possibilities from online cross-tenant placement.

Could equal static allocation simply be too weak a baseline? Table 3 compares against a frozen skewed allocation that helps the initial phase. Once the high-pressure set drifts, however, its overlap falls to 1.14/4; SAKI recovers high-pressure throughput (+17.7% versus frozen skew), reduces compact bytes (-12.3%), and restores overlap to 3.14/4.

RocksDB’s local auto-tuned rate limiter adapts within a tenant’s ceiling but does not move unused budget across tenants. In 640 `static_autotuned` tenant-epochs, every tenant remains at the mid assignment, yielding 0.00/4 overlap; high-pressure P99 worsens by 28.8%, throughput falls by 19.7%, and compaction bytes are essentially unchanged (-0.2%). High tenant-epochs also take 22.0% longer and show 59.7% more drain events. The promoted higher-pressure point repeats the pattern: `static_autotuned` worsens P99 and throughput relative to static, while SAKI improves over it by 32.7% and 59.9%, respectively.

A ranking key that merely chases stale debt would lag the moving high-pressure set. Pressure mode is a backlog-only online key: it improves high-pressure P99 (-14.1%) but lags drift (overlap 1.86/4, max lag 5 windows). Demand mode adds the previous-window offered write signal, raises overlap to 3.00/4, eliminates the 5-window lag, and raises high-pressure throughput from +9.8% to +26.1%. Hybrid mode gives larger tail reductions but still lags (overlap 2.29/4, max lag 2). The ablation matches the design prediction in Section 4.5: backlog symptoms help, but offered demand is what makes the rank-and-assign loop track drift rather than stale debt.

Is the runtime actuator really a maintenance I/O knob? Section 5’s four-tenant microstudy shows limiter traffic rising with the configured budget and compact-plus-flush output tracking it (coverage 0.99–1.02) until maintenance demand saturates near 7.3 MB/s.

Is the result tied to one fragile ranking key? Section 4 treats the formula as an auditable reference ordering, not an optimum. Replaying the 35 post-warmup decisions from the five continuous traces exactly reproduces the online high-budget set; the demand term alone also reproduces the reference top-four set in all 35 decisions. Nearby demand-aware perturbations keep exact or near-exact agreement, whereas a pressure-only negative control matches 10/35 decisions (Table 4). The reference key’s top-four margin is positive throughout (mean 5.30, min 0.39), so the main assignment follows from demand-observable placement under the conserved template rather than from one finely tuned ordering.

Online perturbations on `RUNTIME-DRIFT16` tell the same story. With three paired seeds per row, anchor and residual rescalings keep the headline direction, overlap at 2.86–3.00/4, and max lag at 0–1 window (Table 5). The anchor-only row confirms the runtime demand-only case: demand fixes the high-budget set, while bounded residuals mainly stabilize near-boundary tenants. Thus nearby demand-aware keys keep the same placement behavior; removing demand collapses into the pressure-only case. Because

these rows use three static pairs, row-to-row differences are consistency checks rather than tuning claims.

Finally, would a lower-layer control point be equivalent? On the aligned epoch workload, online `cgroup-v2` control improves high-pressure throughput by 12.5% relative to equal `cgroup` throttling, but the P99 CI crosses zero (-4.4%) and compaction bytes increase by 3.1%. An oracle `cgroup` assignment can express priority when the high-pressure set is known (P99 -9.1%, throughput +15.4%), so the layer is useful as an enforcement envelope. It does not provide the LSM-aware budget-placement signal that SAKI uses.

7.3 Controlled Coverage Around the Anchor Workloads

The next experiments test local coverage around the controlled anchor workloads while preserving the same-budget contract. The epoch sensitivity sweep changes drift speed and high-write pressure around `EPOCH-DRIFT16`. These rows are local coverage checks: the promotion rule keeps capacity-boundary and failed-tenant cases from obscuring the comparison, while the headline claim rests on the repeated anchors above. All promoted points move in the headline direction: high-pressure P99 improves by 6.9–12.9%, high-pressure throughput improves by at least 33.0%, and compaction bytes fall by 17.6–21.3%. The faster-drift point remains CI-strict on high-pressure benefit (P99 -6.9%, throughput +33.9%) with an aggregate-throughput cost (-5.0%), matching the expected responsiveness limit of a previous-window controller.

The continuous workload matrix changes one axis at a time around `RUNTIME-DRIFT16`: 4 KB values, tripled point-read QPS, faster drift, and an 8-tenant cluster. All four promoted variants keep the headline direction: high-pressure P99 improves in every row and high-pressure throughput improves by 20.1–30.4% (Table 6). The boundary is P999 under change-window stress. `VALUE-4K`’s P999 is not statistically separated from static, and `FAST-DRIFT` has P999 regressions in three of five trials despite P99 and throughput wins; the largest regressions coincide with 2/4 or 3/4 overlap before newly high-pressure tenants are promoted. L0-file counts stay zero, pointing to responsiveness rather than L0 buildup.

The window length is also a mechanism setting. Repeating the continuous anchor at 10, 20, and 40 seconds keeps the load, drift distance, and 96 MB/s budget fixed. Only 20 seconds gives a CI-strict high-P99 win; 10 seconds is noisy and raises LOW P99, while 40 seconds lowers overlap to 2.00/4 and leaves lag unbounded in all three seeds.

7.4 Collateral Cost and Liveness

SAKI is a targeted reallocation policy, not a Pareto-improvement claim. Tier-level accounting makes the redistribution explicit. In the continuous aggregate, HIGH tenants receive P99 -27.0%, P999 -47.1%, and write throughput +26.1%; LOW tenants absorb P99 +1.4%, P999 +2.6%, and write throughput -3.0%. The epoch-level pattern is similar: HIGH throughput rises by 38.1%, while MID and LOW throughput change by -9.1% and -1.2%. Total throughput remains close to static in both harnesses, so the benefit is a budget-placement trade-off rather than added capacity.

Read-side and CPU measurements show that bytes/write savings do not hide higher compaction CPU per completed write:

Table 3: Mechanism checks against stronger same-budget baselines. Rows use the comparison named at left, not all rows relative to static.

Comparison	High P99	High tput	Total tput	Compact bytes	Mid tput	Overlap
SAKI vs static	-11.7%	+38.1%	-1.7%	-21.0%	-9.1%	3.14/4
static_biased vs static	-6.1%	+17.4%	-4.2%	-9.9%	-10.0%	1.14/4
static_autotuned vs static	+28.8%	-19.7%	-1.8%	-0.2%	-17.3%	0.00/4
SAKI vs static_biased	-6.0%	+17.7%	+2.8%	-12.3%	+1.1%	3.14 vs 1.14
SAKI vs static_autotuned	-31.5%	+71.9%	+0.3%	-20.8%	+10.0%	3.14 vs 0.00

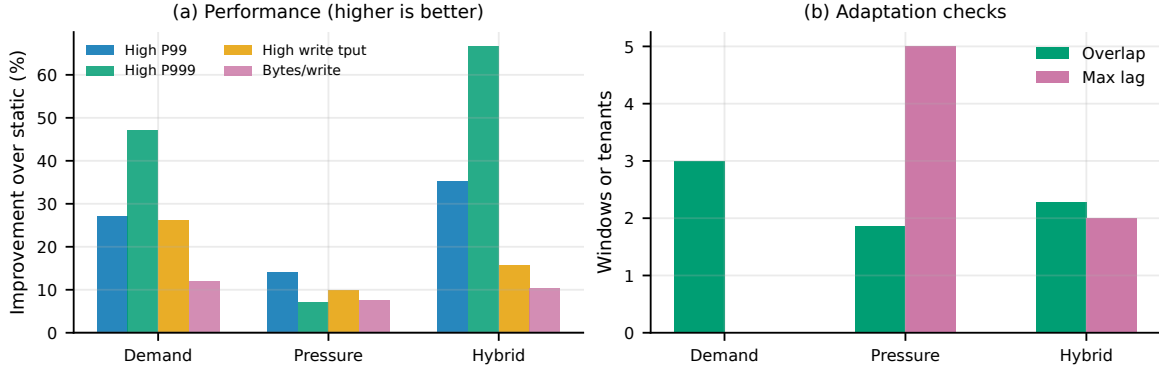


Figure 3: Ranking-key ablation. Demand mode gives strong high-pressure benefit while satisfying adaptation checks; pressure and hybrid keys improve some metrics but lag the moving high-pressure set.

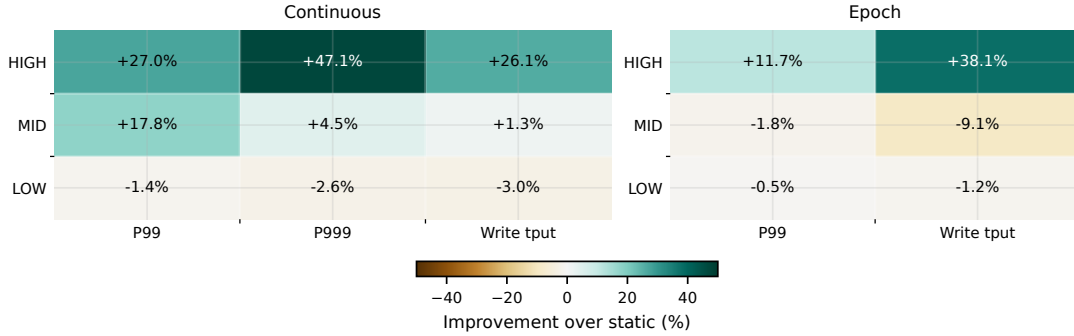


Figure 4: Tier-wise benefit and collateral cost. Cells report SAKI's improvement over static (positive is better), showing the intended high-pressure benefit and the measured redistribution cost to lower tiers.

CPU/write falls by 12.8% overall. Read effects are bounded in this read-light workload (P99 +7.0%, P999 -10.6%, throughput +0.5%) but are collateral accounting, not a read-dominant SLO claim. A separate 360-second run with 16 drift events reports continued high-tenant benefit, overlap 3.00/4, max lag 1 window, and zero tenant failures.

Supplemental 32/64-tenant stresses complete with zero tenant failures and maintain high-set tracking (6/8 and 12/16 overlap). Because this density on the same 80-core host introduces CPU, scheduling, and device-contention confounds, we use these runs only as larger-inventory liveness evidence; full measurements are reported in the artifact.

7.5 Trace Qualification Boundary

The controlled experiments above establish the fixed-budget placement mechanism. Public traces become meaningful controller-validation workloads when they preserve tenant identity and drift, expose actuatable supply under the fixed budget, and create enough RocksDB maintenance stress to exercise compaction-budget placement. We audited two public trace families under the same 11/7/3 MB/s, 112 MB/s budget contract and use them to identify which of those conditions are missing.

As a bridge to public-trace audits, a trace-shaped synthetic stress keeps the required control properties: tenant identity, mixed

Table 4: Ranking-margin replay on 35 post-warmup decisions. Jaccard compares each recomputed top-four set with the reference demand-mode top four.

Ranking key	Exact	Jaccard	True overlap
demand only	35/35	1.00 [1.00,1.00]	3.00/4
anchor $\times 0.5$	31/35	0.95 [0.33,1.00]	2.86/4
anchor $\times 2$	35/35	1.00 [1.00,1.00]	3.00/4
residual $\times 0.5$	35/35	1.00 [1.00,1.00]	3.00/4
residual $\times 2$	31/35	0.95 [0.33,1.00]	2.86/4
pressure-only	10/35	0.61 [0.14,1.00]	2.03/4

reads/writes, uneven rates, hot-set variation, and 20-second drift. SAKI reduces high-pressure P99 by 26.5%, raises throughput by 29.8%, keeps total throughput neutral (-0.1%), and lowers bytes/write by 7.4%, with overlap 3.00/4 and zero failures. Mixed P999 (+21.4%) keeps this as coverage, not production-trace validation.

Meta CacheLib kvcache/202401 [15] preserves drift and replay plumbing but lacks RocksDB maintenance stress: five selected segments completed, yet logical writes reached only 6.7–7.3 MB/s and compaction plateaued near 29–31 MB/s. Meta Tectonic/Baleen storage 0.1% [29] has the complementary problem: scaled aggregate supply can look sufficient, but most windows have only one to three active write tenants, so static-actuable supply is too low. Across 146 candidates, none pass the fixed-budget actuation gate. These audits are therefore trace-qualification results rather than controller comparisons.

8 DISCUSSION AND LIMITATIONS

8.1 Applicability Conditions

SAKI applies when co-located RocksDB instances share a fixed background budget, write pressure drifts at window time scales, and the host can observe tenant-local demand and maintenance symptoms. The evidence supports a narrow systems claim: a host-level rank-and-assign controller can place an existing compaction/I/O budget better than equal sharing under these conditions, without oracle labels or extra capacity.

The bounded-drift condition matters because previous-window observations must remain relevant to the next window. The main workload moves one tenant per 20-second window; FAST-DRIFT pushes to 1.5 tenants/window and still improves P99 and throughput, but its wide P999 interval exposes the expected boundary: newly high-pressure tenants can be hit before the next assignment promotes them. Abrupt reversals, sub-window bursts, and within-window phase changes may outrun the controller. The 20-second window, 4/8/4 split, and concrete budgets are mechanism-isolation settings; Section 4’s conservation rule supports other tier counts, budget spreads, or a continuous allocator. At 32/64 tenants, the controller remains live and tracks high sets, but tail-latency behavior needs tier-spread calibration and platform isolation from CPU and scheduling contention.

8.2 Deployment Assumptions

The continuous harness changes background rate limits through `RateLimiter::SetBytesPerSecond()`. The measured rate-limiter coverage microstudy shows this knob shapes flush/compaction I/O in our configuration, so foreground latency and throughput changes arise indirectly from maintenance service rather than direct foreground throttling. RocksDB’s built-in auto-tuned rate limiter remains useful local machinery, but it adapts within per-tenant ceilings; SAKI supplies the host-level loop that moves unused budget across tenants. A natural deployment variant is to let SAKI choose tenant ceilings while RocksDB autotuning adapts inside each ceiling; we did not evaluate that combined policy.

A production deployment would need tenant inventory, periodic RocksDB statistics, a conserved host budget, and per-tenant runtime actuation. Offered demand should be measured before backpressure or retries hide pressure, and operators would likely add smoothing, hysteresis, cooldowns, dwell times, and SLO-aware floors. If metrics are incomplete, the safe fallback is all-mid sharing. Deployments should treat the reference key as one demand-aware ordering, normalize local signals, and bound residual terms while preserving budget conservation and demand observability. Other LSM engines need analogous tenant-scoped maintenance signals and a runtime maintenance-budget knob.

8.3 Resource and Fairness Boundaries

SAKI controls one resource dimension: background I/O budget placement. CPU, cache, memory, device queueing, and per-level write amplification may dominate elsewhere, especially when limiter traffic stops tracking maintenance output or other metrics saturate while compaction debt remains high. A full maintenance-cost decomposition is a separate multi-resource control problem.

The policy is also a targeted reallocation: shifting budget to high-pressure tenants can cost lower-pressure tenants. The measured LOW-tier costs and the 4 KB-value P999 boundary motivate SLO-aware floors and fairness policy around the same fixed-budget loop. Scan-heavy workloads, range-scan SLOs, and read-dominant deployments require their own ranking and validation layer.

8.4 Trace Validation Boundary

The public-trace audits define the conditions required for meaningful controller validation. CacheLib validates trace-shaped replay but creates too little RocksDB maintenance stress, while Baleen often has too few active write tenants per window to actuate the fixed all-mid budget. The next validation layer needs storage-engine traces with tenant identity, drift, and actuable supply under the same fixed-budget contract; non-leveled compaction styles and joint multi-resource control are natural extensions once that condition is met.

9 RELATED WORK

LSM compaction and tuning. The original LSM-tree design buffers updates and later merges them into disk-resident sorted components [20]. The survey by Luo and Carey describes the resulting design space [14], and Sarkar et al. formalize compaction triggers, layout, granularity, and data movement as design primitives [22]. Monkey and Dostoevsky optimize Bloom-filter memory, merge

Table 5: Ranking-key robustness on RUNTIME-DRIFT16. Ablation rows use $n=3$ paired seeds and perturb one weight group; cells are mean [min, max] percent of static. The $\star$ row is the published $n=5$ main result.

Perturbation	High P99	High tput	Overlap	Max lag	Top-4 Jaccard
anchor $\times 0.5$ (3.00)	-30.3% [-38.1,-23.6]	+22.4% [+18.9,+27.1]	2.90/4	1	0.94 [0.88,1.00]
anchor $\times 2$ (12.00)	-27.5% [-42.4,-18.2]	+23.7% [+22.2,+26.2]	3.00/4	0	1.00 [1.00,1.00]
residual $\times 0.5$	-30.0% [-39.8,-16.7]	+24.3% [+21.3,+28.2]	3.00/4	0	1.00 [1.00,1.00]
residual $\times 2$	-35.8% [-49.6,-25.7]	+25.0% [+20.9,+27.1]	2.86/4	1	0.94 [0.89,1.00]
anchor only (residual=0)	-31.9% [-40.2,-26.2]	+21.5% [+12.6,+26.0]	3.00/4	0	1.00 [1.00,1.00]
fixed SAKI ($n=5$, paper main) $\star$	-27.0%	+26.1%	3.00/4	0	1.00 (self)

Table 6: Local workload matrix, five trials per variant, online vs. static (percent changes, 95% CI half-widths). Zero failed tenants.

Variant	High P99	High P999	High tput	Total tput	Bytes/write	Overlap
VALUE-4K	-28.6% $\pm$ 16.2	-7.1% $\pm$ 20.6	+20.1% $\pm$ 4.7	+0.6% $\pm$ 1.5	-9.0% $\pm$ 3.0	3.00/4
READ-3X	-15.4% $\pm$ 17.5	-38.4% $\pm$ 27.6	+30.4% $\pm$ 7.6	-0.2% $\pm$ 4.0	-10.2% $\pm$ 5.8	3.00/4
FAST-DRIFT	-17.6% $\pm$ 10.1	-1.1% $\pm$ 46.6	+20.5% $\pm$ 10.3	+4.4% $\pm$ 3.4	-3.3% $\pm$ 4.4	2.57/4
TENANT-8	-42.2% $\pm$ 12.7	-52.6% $\pm$ 16.9	+28.7% $\pm$ 5.8	+7.0% $\pm$ 1.9	-11.3% $\pm$ 3.4	1.57/2

Table 7: Window-length sensitivity around the continuous anchor ($n=3$ per row, online vs. static; CI half-widths).

Window	High P99	High tput	Overlap	LOW P99
10 s	+6.8% $\pm$ 39.2	+23.4% $\pm$ 58.1	3.47/4	+110.5% $\pm$ 367.8
20 s	-26.9% $\pm$ 14.0	+16.1% $\pm$ 8.7	3.00/4	+24.6% $\pm$ 63.6
40 s	-14.4% $\pm$ 44.8	+14.5% $\pm$ 19.8	2.00/4	-36.3% $\pm$ 84.2

policy, and space-time tradeoffs [2, 3]; WiscKey and PebblesDB change value placement or merge structure [13, 21]. SILK schedules internal LSM I/O to reduce latency spikes, while ADOC, vLSM, and ArceKV tune dataflow, LSM shape, or compaction thresholds for a single engine or workload [1, 12, 30, 31]. Dremel adaptively tunes RocksDB parameters with an online bandit on a single instance [33], and Dong et al.’s RocksDB retrospective at Meta tracks how amplification priorities shifted with hardware and workload, still inside one engine [5]. SAKI is orthogonal: it leaves each engine’s compaction policy and parameters unchanged, treats these single-engine contributions as the inner control loop, and asks how a fixed host budget should be divided across co-located instances as demand drifts. The two control layers can be combined.

Multi-tenant storage control. Argon, PARDA, mClock, Pisces, Cake, and IOFlow allocate or route storage service across shared servers, hypervisors, and software-defined storage planes [7, 8, 23, 25–27]. Linux cgroups and IOCost provide lower-layer block-I/O weighting [9, 24], and Gimbal extends the line to SmartNIC JBOFs that schedule co-tenant NVMe traffic on the storage data path [18]. These systems enforce fairness or SLOs over I/O streams but do not observe LSM debt signals or choose which RocksDB instance receives background compaction service. SAKI uses those engine-level signals and treats kernel or network-side I/O control as a companion enforcement layer rather than the placement mechanism itself.

Multi-tenant LSM deployments. Recent KV-store work moves isolation closer to the storage engine. FlexEngine uses admission control for multi-tenant serverless LSM deployments, Iso-KVSSD and ZenFS+ use device namespace or zoned-placement mechanisms for isolation, and Delta Fair Sharing proposes in-engine multi-tenant fairness for RocksDB [6, 17, 19, 28]. CaaS-LSM and DFlush move compaction or flush work into disaggregated or DPU-assisted settings [4, 32]. SAKI instead targets the common deployment where tenants are separate, unmodified RocksDB instances on one host and the available background budget is fixed.

RocksDB controls. RocksDB exposes per-instance options, statistics, event listeners, and rate limiting [11, 16]. The auto-tuned rate limiter solves local adaptation within a ceiling; SAKI supplies the host-level rank-and-assign loop that places budgets across instances under a conserved aggregate budget. Public CacheLib and Baleen traces inform our trace-qualification audits, but cache admission, prefetching, and request-trace replay are separate from the fixed-budget compaction-placement problem [15, 29].

10 CONCLUSION

SAKI addresses multi-tenant RocksDB compaction-debt mismatch with an external conserved rank-and-assign controller that reallocates per-tenant background budgets from previous-window observations while preserving the aggregate host budget. A demand-observability model explains why demand-aware ranking keys track bounded drift and why pressure-only ranking can chase stale debt. Repeated epoch and continuous runs show high-pressure write-tail and throughput gains; stronger baselines, ranking-key ablations, ranking-margin replay, cgroup-v2, and rate-limiter coverage checks tie those gains to host-level placement of a real maintenance-I/O knob. Workload variants, collateral accounting, and trace qualification define the current evidence boundary. For bounded write-heavy drift under fixed shared maintenance budgets, SAKI shows that shared compaction budgeting is not merely a configuration detail but a host-level control point.

REFERENCES

- [1] Oana Balmau, Diego Didona, Rachid Guerraoui, Willy Zwaenepoel, Huapeng Yuan, Aakash Arora, Karan Gupta, and Pavan Konka. 2019. SILK: Preventing Latency Spikes in Log-Structured Merge Key-Value Stores. In *2019 USENIX Annual Technical Conference (USENIX ATC '19)*. USENIX Association, Renton, WA, USA, 753–766.
- [2] Niv Dayan, Manos Athanassoulis, and Stratos Idreos. 2017. Monkey: Optimal Navigable Key-Value Store. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD '17)*. Association for Computing Machinery, New York, NY, USA, 79–94. <https://doi.org/10.1145/3035918.3064054>
- [3] Niv Dayan and Stratos Idreos. 2018. Dostoevsky: Better Space-Time Trade-Offs for LSM-Tree Based Key-Value Stores via Adaptive Removal of Superfluous Merging. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 505–520. <https://doi.org/10.1145/3183713.3196927>
- [4] Chen Ding, Kai Lu, Quanyi Zhang, Zekun Ye, Ting Yao, Daohui Wang, Huatao Wu, and Jiguang Wan. 2025. DFlush: DPU-Offloaded Flush for Disaggregated LSM-based Key-Value Stores. *Proceedings of the ACM on Management of Data* 3, 3, Article 147 (2025), 28 pages. <https://doi.org/10.1145/3725284>
- [5] Siying Dong, Andrew Kryczka, Yanqin Jin, and Michael Stumm. 2021. RocksDB: Evolution of Development Priorities in a Key-value Store Serving Large-scale Applications. *ACM Transactions on Storage* 17, 4, Article 26 (October 2021), 32 pages. <https://doi.org/10.1145/3483840>
- [6] Tyler Griggs, Soujanya Ponnappalli, Dev Bali, Wenjie Ma, James DeLoye, Audrey Cheng, Jaewan Hong, Natacha Crooks, Scott Shenker, Ion Stoica, and Matei Zaharia. 2026. Delta Fair Sharing: Performance Isolation for Multi-Tenant Storage Systems. arXiv:2601.20030. <https://doi.org/10.48550/arXiv.2601.20030>
- [7] Ajay Gulati, Irfan Ahmad, and Carl A. Waldspurger. 2009. PARD: Proportional Allocation of Resources for Distributed Storage Access. In *7th USENIX Conference on File and Storage Technologies (FAST '09)*. USENIX Association, San Francisco, CA, USA, 85–98.
- [8] Ajay Gulati, Arif Merchant, and Peter J. Varman. 2010. mClock: Handling Throughput Variability for Hypervisor IO Scheduling. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI '10)*. USENIX Association, Vancouver, BC, Canada, 437–450.
- [9] Tejun Heo, Dan Schatzberg, Andrew Newell, Song Liu, Saravanan Dhakshinamurthy, Iyswarya Narayanan, Josef Bacik, Chris Mason, Chunqiang Tang, and Dimitrios Skarlatos. 2022. IOCost: Block IO Control for Containers in Datacenters. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 595–608. <https://doi.org/10.1145/3503222.3507727>
- [10] Shubham Kaushik and Subhadeep Sarkar. 2024. Anatomy of the LSM Memory Buffer: Insights & Implications. In *Proceedings of the 10th International Workshop on Testing Database Systems (DBTest '24)*. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3662165.3662766>
- [11] Andrew Kryczka. 2017. Auto-tuned Rate Limiter. <https://rocksdb.org/blog/2017/12/18/17-auto-tuned-rate-limiter.html>. RocksDB blog, accessed 2026-05-14.
- [12] Junfeng Liu, Haoxuan Xie, and Siqiang Luo. 2026. ArceKV: Towards Workload-driven LSM-compactions for Key-Value Store Under Dynamic Workloads. *Proceedings of the VLDB Endowment* 19, 5 (2026), 958–972. <https://doi.org/10.14778/3796195.3796208>
- [13] Lanyue Lu, Thanumalayan Sankaranarayanan Pillai, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2016. WiscKey: Separating Keys from Values in SSD-conscious Storage. In *14th USENIX Conference on File and Storage Technologies (FAST '16)*. USENIX Association, Renton, WA, USA, 133–148.
- [14] Chen Luo and Michael J. Carey. 2020. LSM-based Storage Techniques: A Survey. *The VLDB Journal* 29, 1 (2020), 393–418.
- [15] Meta Platforms, Inc. 2024. CacheLib Public Cache Trace Dataset. https://github.com/cacheMon/cache_dataset. Meta CacheLib kvcache/202401 traces, accessed 2026-05-18.
- [16] Meta Platforms, Inc. 2026. RocksDB. <https://rocksdb.org/>. Accessed 2026-05-14.
- [17] Donghyun Min and Youngjae Kim. 2021. Isolating Namespace and Performance in Key-Value SSDs for Multi-tenant Environments. In *Proceedings of the 36th Annual ACM Symposium on Applied Computing*. Association for Computing Machinery, New York, NY, USA, 1112–1121. <https://doi.org/10.1145/3412841.3442008>
- [18] Jaehong Min, Ming Liu, Tapan Chugh, Chenxingyu Zhao, Patrick Wei, In Hwan Doh, and Arvind Krishnamurthy. 2021. Gimbal: Enabling Multi-tenant Storage Disaggregation on SmartNIC JBOfs. In *Proceedings of the 2021 ACM SIGCOMM Conference (SIGCOMM '21)*. Association for Computing Machinery, New York, NY, USA, 106–122. <https://doi.org/10.1145/3452296.3472940>
- [19] Myoungsoon Oh, Seehwan Yoo, Jongmoo Choi, J. Park, and C. E. Choi. 2023. ZenFS+: Nurturing Performance and Isolation to ZenFS. *IEEE Access* 11 (2023), 26344–26357. <https://doi.org/10.1109/ACCESS.2023.3257354>
- [20] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. 1996. The Log-Structured Merge-Tree (LSM-Tree). *Acta Informatica* 33, 4 (1996), 351–385. <https://doi.org/10.1007/s002360050048>
- [21] Pandian Raju, Rohan Kadekodi, Vijay Chidambaram, and Ittai Abraham. 2017. PebblesDB: Building Key-Value Stores using Fragmented Log-Structured Merge Trees. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP '17)*. Association for Computing Machinery, New York, NY, USA, 497–514. <https://doi.org/10.1145/3132747.3132765>
- [22] Subhadeep Sarkar, Dimitris Staratzis, Zichen Zhu, and Manos Athanassoulis. 2021. Constructing and Analyzing the LSM Compaction Design Space. *Proceedings of the VLDB Endowment* 14, 11 (2021), 2216–2229. <https://doi.org/10.14778/3476249.3476274>
- [23] David Shue, Michael J. Freedman, and Anees Shaikh. 2012. Performance Isolation and Fairness for Multi-Tenant Cloud Storage. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI '12)*. USENIX Association, Hollywood, CA, USA, 349–362.
- [24] The Linux Kernel Developers. 2026. Control Group v2. <https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html>. Accessed 2026-05-15.
- [25] Eno Thereska, Hitesh Ballani, Greg O'Shea, Thomas Karagiannis, Ant Rowstron, Tom Talpey, Richard Black, and Timothy Zhu. 2013. IOFlow: A Software-Defined Storage Architecture. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (SOSP '13)*. Association for Computing Machinery, New York, NY, USA, 182–196. <https://doi.org/10.1145/2517349.2522723>
- [26] Matthew Wachs, Michael Abd-El-Malek, Eno Thereska, and Gregory R. Ganger. 2007. Argon: Performance Isolation for Shared Storage Servers. In *5th USENIX Conference on File and Storage Technologies (FAST '07)*. USENIX Association, San Jose, CA, USA, 61–76.
- [27] Andrew Wang, Shivaram Venkataraman, Sara Alspaugh, Randy H. Katz, and Ion Stoica. 2012. Cake: Enabling High-Level SLOs on Shared Storage Systems. In *Proceedings of the Third ACM Symposium on Cloud Computing (SoCC '12)*. Association for Computing Machinery, New York, NY, USA, Article 14, 14 pages. <https://doi.org/10.1145/2391229.2391243>
- [28] Yingjia Wang, Caixin Gong, Guoyun Zhu, Sheng Wang, Zhengheng Wang, Huan Liu, Junzhi Shi, Wu Qin, Wei Zhang, Feifei Li, and Ming-Chang Yang. 2026. Making LSM-Tree-based Key-Value Store Practical and Efficient for Multi-Tenant Serverless Cloud Databases. *Proceedings of the ACM on Management of Data* 4, 1, Article 53 (February 2026), 25 pages. <https://doi.org/10.1145/3786667>
- [29] Daniel Lin-Kit Wong, Hao Wu, Carson Molder, Sathya Gunasekar, Jimmy Lu, Snehal Khandkar, Abhinav Sharma, Daniel S. Berger, Nathan Beckmann, and Gregory R. Ganger. 2024. Baleen: ML Admission & Prefetching for Flash Caches. In *22nd USENIX Conference on File and Storage Technologies (FAST '24)*. USENIX Association, Santa Clara, CA, 347–371. <https://www.usenix.org/conference/fast24/presentation/wong>
- [30] Giorgos Xanthakis, Antonios Katsarakis, Giorgos Saloustros, and Angelos Bilas. 2024. vLSM: Low Tail Latency and I/O Amplification in LSM-based KV Stores. arXiv:2407.15581. <https://doi.org/10.48550/arXiv.2407.15581>
- [31] Jinghuan Yu, Sam H. Noh, Young-ri Choi, and Chun Jason Xue. 2023. ADOC: Automatically Harmonizing Dataflow Between Components in Log-Structured Key-Value Stores for Improved Performance. In *21st USENIX Conference on File and Storage Technologies (FAST '23)*. USENIX Association, Santa Clara, CA, USA, 65–80.
- [32] Qiaolin Yu, Chang Guo, Jay Zhuang, Viraj Thakkar, Jianguo Wang, and Zhichao Cao. 2024. CaaS-LSM: Compaction-as-a-Service for LSM-based Key-Value Stores in Storage Disaggregated Infrastructure. *Proceedings of the ACM on Management of Data* 2, 3, Article 124 (2024), 28 pages. <https://doi.org/10.1145/3654927>
- [33] Chenxingyu Zhao, Tapan Chugh, Jaehong Min, Ming Liu, and Arvind Krishnamurthy. 2022. Dremel: Adaptive Configuration Tuning of RocksDB KV-Store. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 6, 2, Article 33 (2022), 30 pages. <https://doi.org/10.1145/3530903>